Alexandria University
Faculty of Engineering
Computer and Communication Program

CC484 Pattern Recognition

Assignment Three: Speech Emotion Recognition

Project Link: [Colab Notebook]

Name	ID
Omar Walied Mohamed	7058
Hend Khaled Aly	6986
Habiba Mohamed Hefny	6939

Table of Contents

- **What is Speech Emotion Recognition?**

Speech emotion recognition (SER) refers to the process of automatically identifying and classifying emotions from speech signals. It involves using computational techniques to analyze acoustic features extracted from the speech signal and determining the emotional state of the speaker.

Speech carries various cues that can indicate the emotional state of a person. These cues include prosodic features such as pitch, rhythm, and intonation, as well as spectral features related to the frequency content of the speech signal. By analyzing these features, SER systems aim to recognize and classify emotions such as happiness, sadness, anger, fear, and more.

The process of speech emotion recognition typically involves the following steps:

Preprocessing: The speech signal is processed to remove noise, normalize the volume, and segment it into smaller units, such as frames or phonemes.

Feature Extraction: Acoustic features are extracted from the preprocessed speech signal. These features can include pitch contour, energy, spectral characteristics, and prosodic features.

Feature Selection: Relevant features are selected to reduce dimensionality and remove irrelevant or redundant information. This step helps improve the accuracy and efficiency of emotion classification.

Emotion Classification: Machine learning or pattern recognition techniques are applied to classify the extracted features into different emotion categories. Common classification algorithms include support vector machines (SVMs), hidden Markov models (HMMs), artificial neural networks (ANNs), and decision trees.

Evaluation: The performance of the SER system is assessed using evaluation metrics such as accuracy, precision, recall, and F1 score. This step helps gauge the effectiveness of the system and identify areas for improvement.

Speech emotion recognition has various applications, including human-computer interaction, affective computing, psychological research, and social robotics. It can be used in systems that aim to understand and respond to users' emotions, improve human-robot interaction, or analyze emotional states in clinical or therapeutic settings.

- **Problem Description:**

  First. We imported the libraries necessary for the project

  Second. Categorize the audio files in the specified directory based on their emotions, where we define a list of different emotions such as anger, disgust, fear, happiness, neutral, and sadness. The emotion is extracted from the file name by accessing the third element in the file name, and after the extraction, the code checks if the extracted emotion is valid or not.

  Third. Define a function called calculate_energy to calculate the energy of an audio frame by squaring the audio samples, summing the squared values, and then dividing by the frame length to obtain the normalized energy per frame.

  Fourth. Create an empty dataframe, specify data types for data frame columns, loop over each emotion list, get the file paths for each emotion, load and process each audio file using librosa, extract spectrogram, convert spectogram to PIL image, compute various audio features such as MFCC, STFT, ZCR, F0 and spectral centroid, then normalize each of those features.

Fifth. Concatenate features vectors, map the emotion label, append data to data frame, plot the waveform of each audio signal.

Sixth. Retrieve spectrogram images from a data frame, resize them to a consistent shape, converts them to NumPy arrays, reshapes them if necessary, and stores them along with their corresponding emotion labels in separate NumPy arrays.

Seventh. Create a label encoder to convert the categorical labels into numeric values that can be processed. Each unique emotion is assigned a unique numeric value.

Eighth. Split data into training, validation, and testing sets, then ensure that the data is divided in a startified manner, preserving the distribution of emotion classes in each split, and allows further separation of specific columns if needed.

Ninth. Define a convolutional neural network (CNN) model for spectrogram data using keras API from TensorFlow, the model is defined with convolutional, pooling, dropout, and dense layers for processing spectrogram data . The model is designed to learn and classify the input spectrograms into one of six possible classes.

Tenth. Set up a callback function called ModelCheckpoint in Keras, which is used to save the best model during training. the model will continuously monitor the validation accuracy during training and save the model weights to the specified file path whenever the validation accuracy improves. This ensures that you have access to the best-performing model at the end of training.

Eleventh. Compile and trains the model using the training and validation data. The training process will iterate over the specified number of epochs, updating the model's weights based on the optimization algorithm and minimizing the loss function. The training progress and evaluation metrics will be stored in the

history object, which can be used for further analysis and visualization.

Twelfth. Evaluate the performance of the trained model on the test data and calculates various classification metrics: precision, recall, and F1 score.

Thirteenth. Evaluate the performance of the trained CNN model on the test data and visualizes the confusion matrix to gain insights into the model's predictions.

Fourteenth. Identify the most confusing classes based on the confusion matrix computed earlier. Then perform data splitting into training/validation and testing sets.

Fifteenth. Set up a CNN model with multiple convolutional and dense layers for a specific input shape and number of output classes. By compiling the model and displaying a summary of its architecture for further training and evaluation. Then convert the training and validation data from their original formats to NumPy arrays, perform string operations and type conversion to process the data elements, and reshape the target variables for compatibility with further processing and modeling.

Sixteenth. Compare the predicted labels generated by a model with the true labels from the test set.

Seventeenth. Evaluates the performance of the predicted labels by computing the accuracy, F1 score, and confusion matrix and display the results.

Eighteenth. Generate a heatmap visualization of the confusion matrix and identify the most confusing classes based on the confusion matrix.

- **Code Explaination and Screenshots:**

```
from google.colab import drive
drive.mount('/content/drive/')
```

```
Drive already mounted at /content/drive/; to attempt to forcibly remount,
```

```
import os
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf
from tensorflow.keras import layers, models
```

```
#classifying according to emotion


# Define a list of emotions
emotions = ['ANG', 'DIS', 'FEA', 'HAP', 'NEU', 'SAD']

# Path to directory containing audio files
audio_path = '/content/drive/MyDrive/PR/Crema'

# Create a dictionary to store the file paths for each emotion
emotion_files = {e: [] for e in emotions}
# Loop over each audio file in directory
for audio_file_name in os.listdir(audio_path):
```

```
# Loop over each audio file in directory
for audio_file_name in os.listdir(audio_path):

    # Check if file is a WAV file
    if audio_file_name.endswith('.wav'):
        # Extract the emotion from file name
        file_parts = audio_file_name.split('_')
        emotion = file_parts[2]

        # Check if the emotion is valid
        if emotion in emotions:
            # Construct path to audio file
            audio_file_path = os.path.join(audio_path, audio_file_name)

            # Add the file path to the list of files for this emotion
            emotion_files[emotion].append(audio_file_path)

# Print the number of files for each emotion
#for emotion, files in emotion_files.items():
 #  print(f'{emotion}: {len(files)}')
```

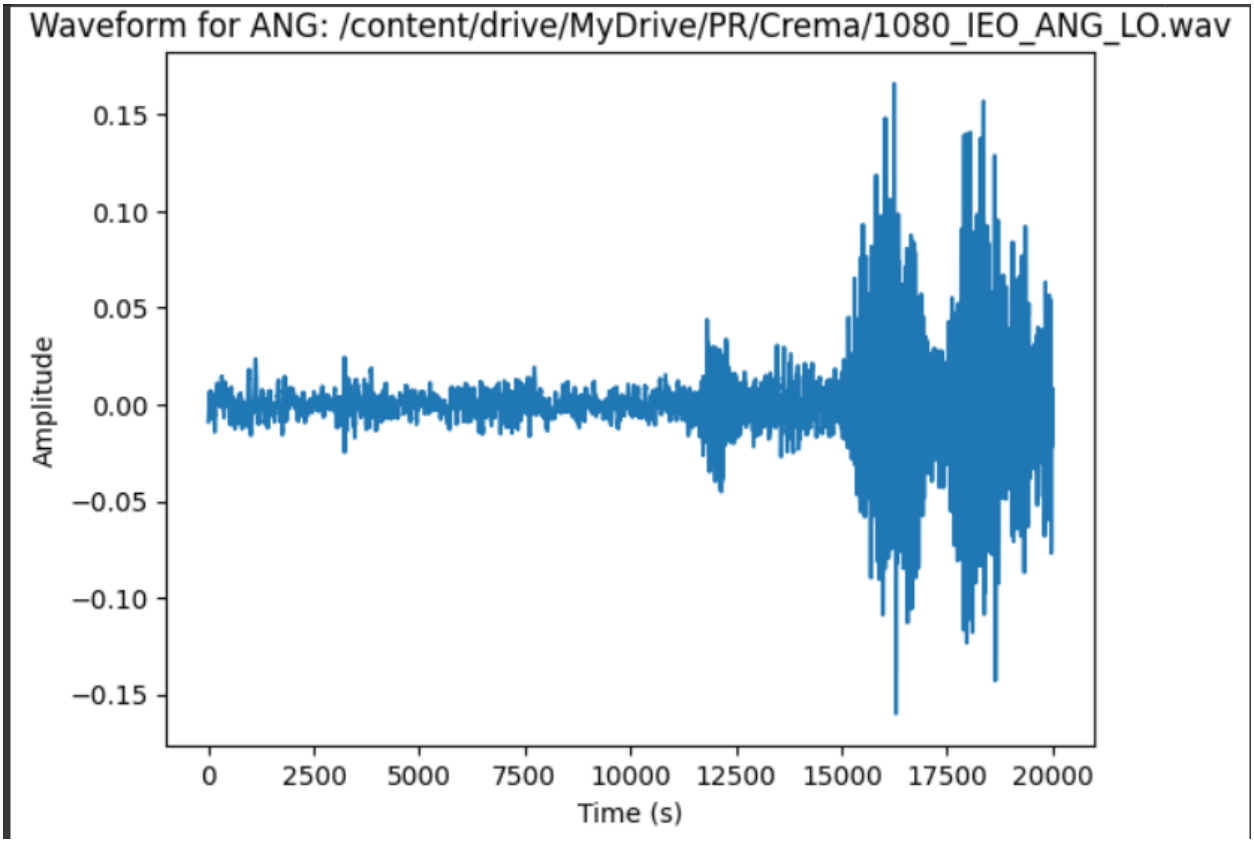
```
def calculate_energy(audio_array, frame_length):
    squared_values = np.square(audio_array)
    sum_of_squares = np.sum(squared_values)
    normalized_energy = sum_of_squares / frame_length
    return normalized_energy
```

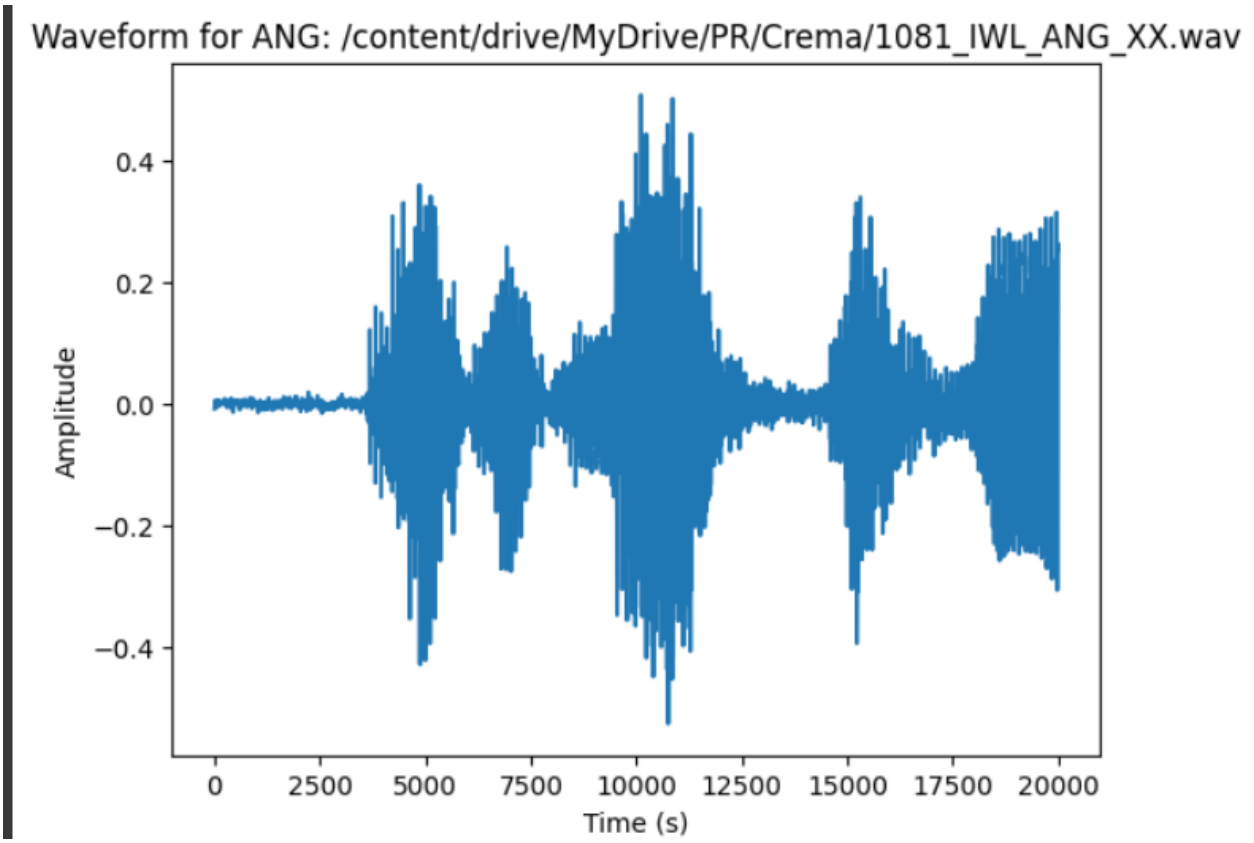
```python
from librosa.feature.spectral import zero_crossing_rate
import os
import pandas as pd
import numpy as np
import librosa
from PIL import Image


# Path to directory containing audio files
audio_path = '/content/drive/MyDrive/PR/Crema'

data = pd.DataFrame(columns=['emotion', 'mel_spec_db', 'features1'])
types1={'mel_spec_db':float}
data.astype(types1)

smallest=100


# Loop over each emotion in list
for emotion in emotions:
    # Get the file paths for this emotion
    emotion_file = [os.path.join(audio_path, f) for f in os.listdir(audio_path) if f.endswith('.wav') and f.split('_')[2] == emotion]
    # Load and plot the audio for this emotion

    for file_path in emotion_file:

        # Load audio file
        audio, sr = librosa.load(file_path, sr=None)



        # Extract spectrogram
        spectrogram = librosa.stft(audio)
        spectrogram_db = librosa.amplitude_to_db(np.abs(spectrogram))
```

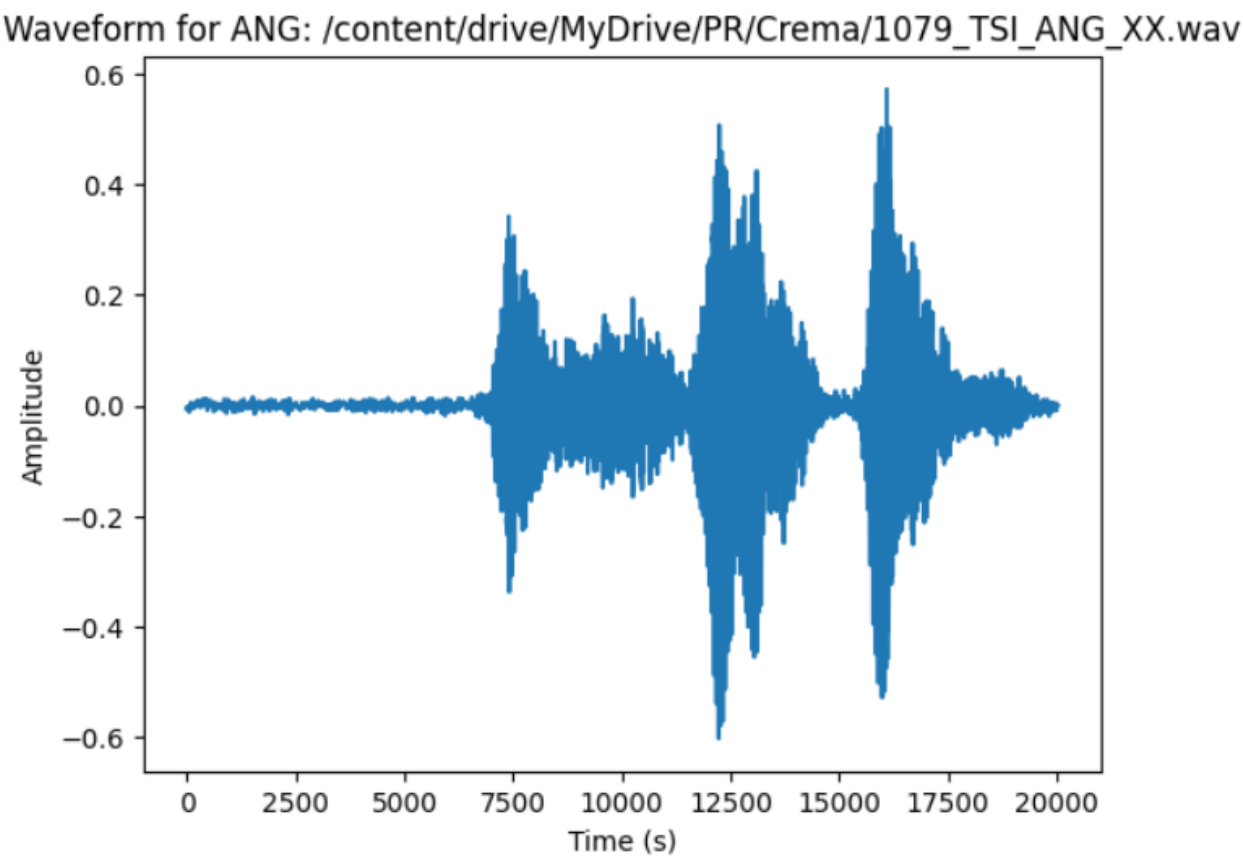
```python
# Convert spectrogram to PIL Image
spectrogram_image = Image.fromarray(spectrogram_db)


# audio = np.concatenate([audio,audio,audio,audio])
audio=audio[:20000]

mfcc = librosa.feature.mfcc(y=audio,sr=sr)
stft = np.abs(librosa.stft(audio))
zcr = librosa.feature.zero_crossing_rate(audio)
f0 = librosa.yin(audio, fmin=80, fmax=400, sr=sr)
f0=f0.reshape(1,f0.shape[0])
spectral_centroids = librosa.feature.spectral_centroid(y=audio, sr=sr)

mfcc_normalized = (mfcc - np.mean(mfcc)) / np.std(mfcc)
stft_normalized = (stft - np.mean(stft)) / np.std(stft)
zcr_normalized = (zcr - np.mean(zcr)) / np.std(zcr)
f0_normalized = (f0 - np.mean(f0)) / np.std(f0)
spectral_centroids_normalized = (spectral_centroids - np.mean(spectral_centroids)) / np.std(spectral_centroids)



features1=np.concatenate((mfcc_normalized.flatten(),stft_normalized.flatten(),zcr_normalized.flatten(),f0_normalized.flatten(),spectral_centroids_normalized.flatten()),axis=0)



frame_length = len(audio)
energy = calculate_energy(audio, frame_length)

temp='no'
['ANG', 'DIS', 'FEA', 'HAP', 'NEU', 'SAD']
```

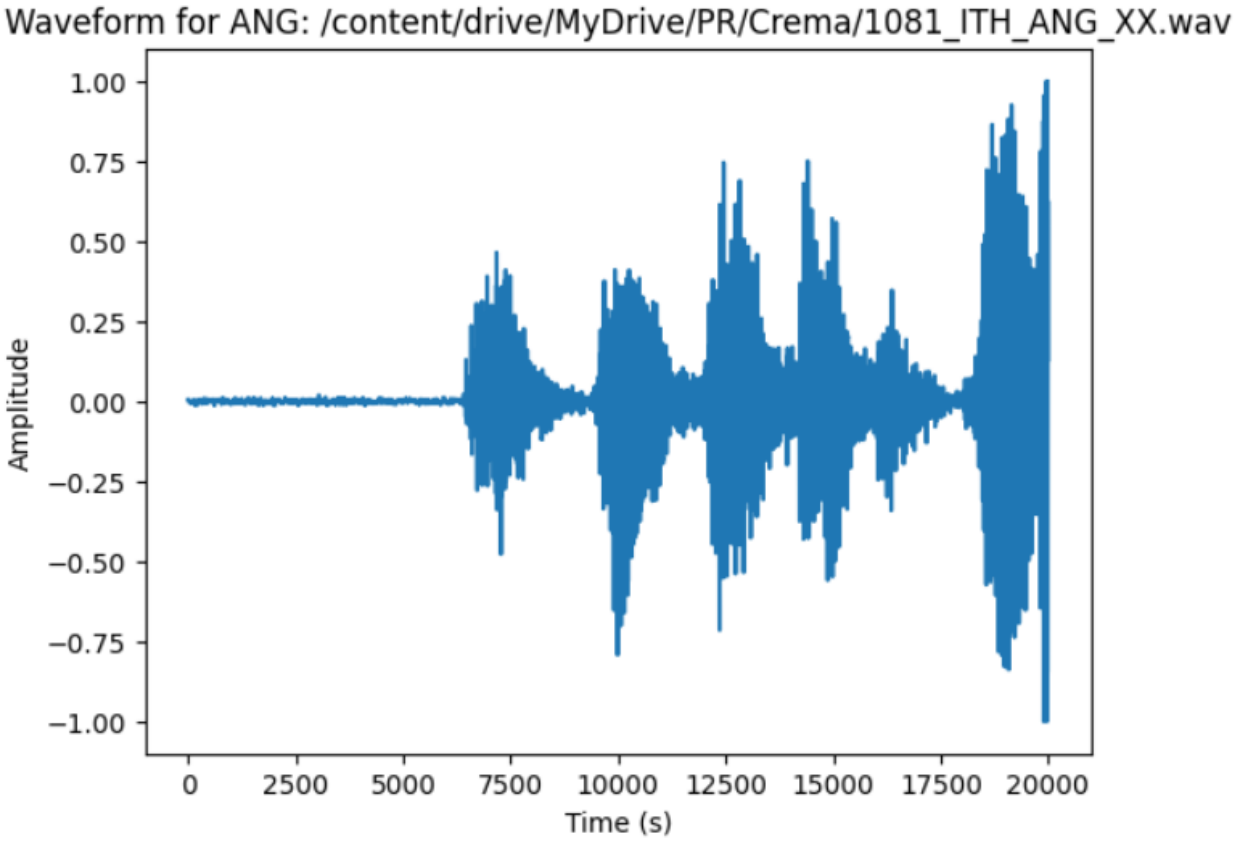
```python
for j in range(6):
  if emotion==emotions[j]:
    temp=j

np.set_printoptions(threshold=np.inf)
data = data.append({'emotion': temp, 'mel_spec_db': spectrogram_image, 'features1': np.array2string(features1, separator=',',threshold=np.inf, prefix='[', suffix=']')}, ignore_index
```

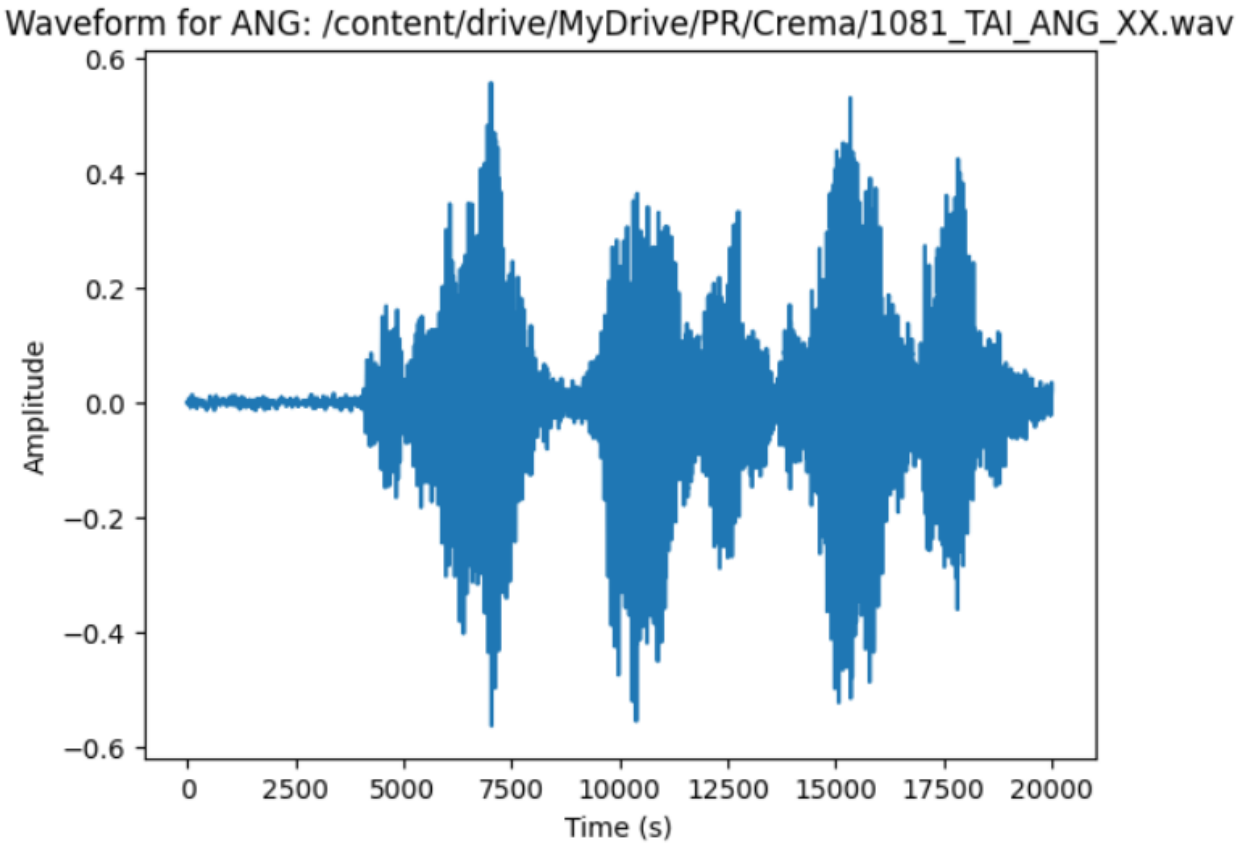
```python
Plot waveform
plt.figure()
plt.title(f'Waveform for {emotion}: {file_path}')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.plot(audio)
plt.show()
```

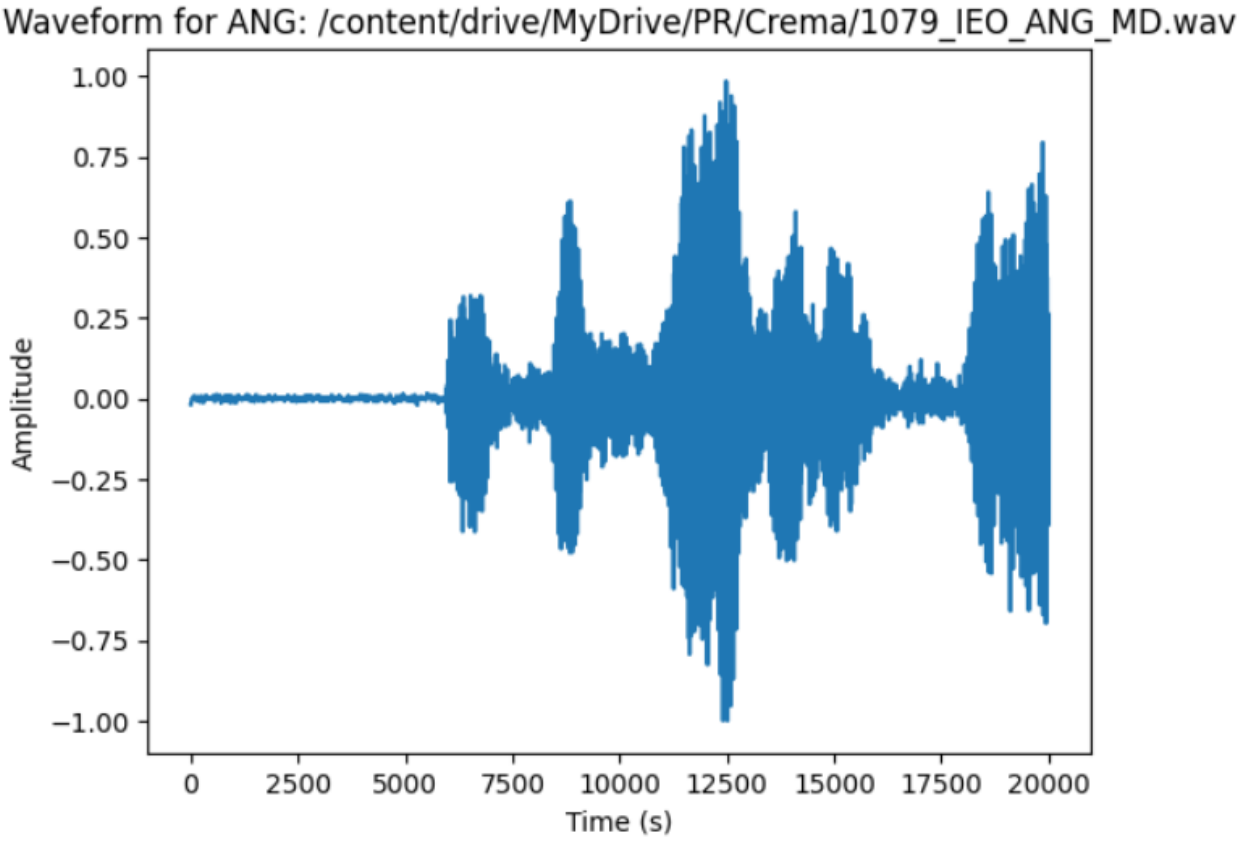

Waveform for ANG: /content/drive/MyDrive/PR/Crema/1080_IEO_ANG_LO.wav
Amplitude
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
0
2500
5000
7500
10000
12500
15000
17500
20000
Time (s)

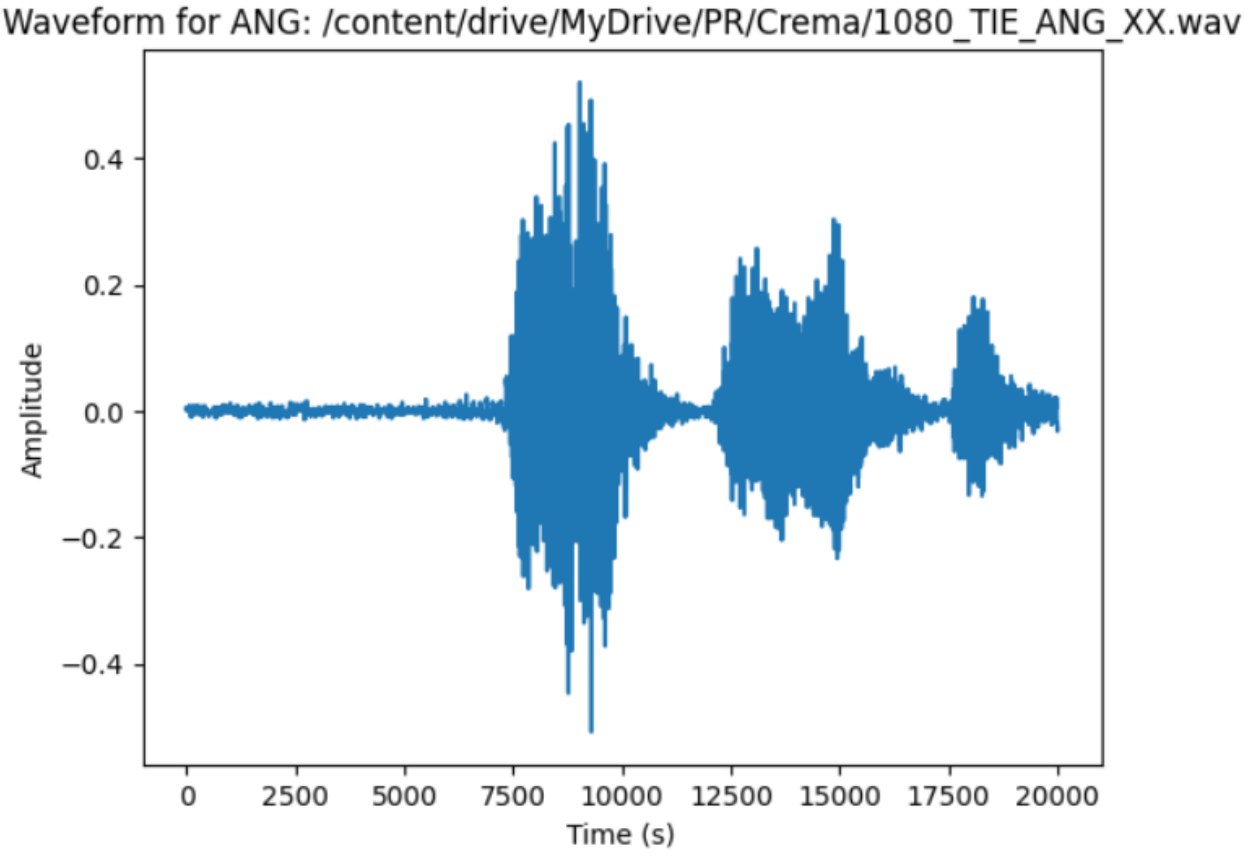

Waveform for ANG: /content/drive/MyDrive/PR/Crema/1081_IWL_ANG_XX.wav
Amplitude
0.4
0.2
0.0
−0.2
−0.4
0
2500
5000
7500
10000
12500
15000
17500
20000
Time (s)

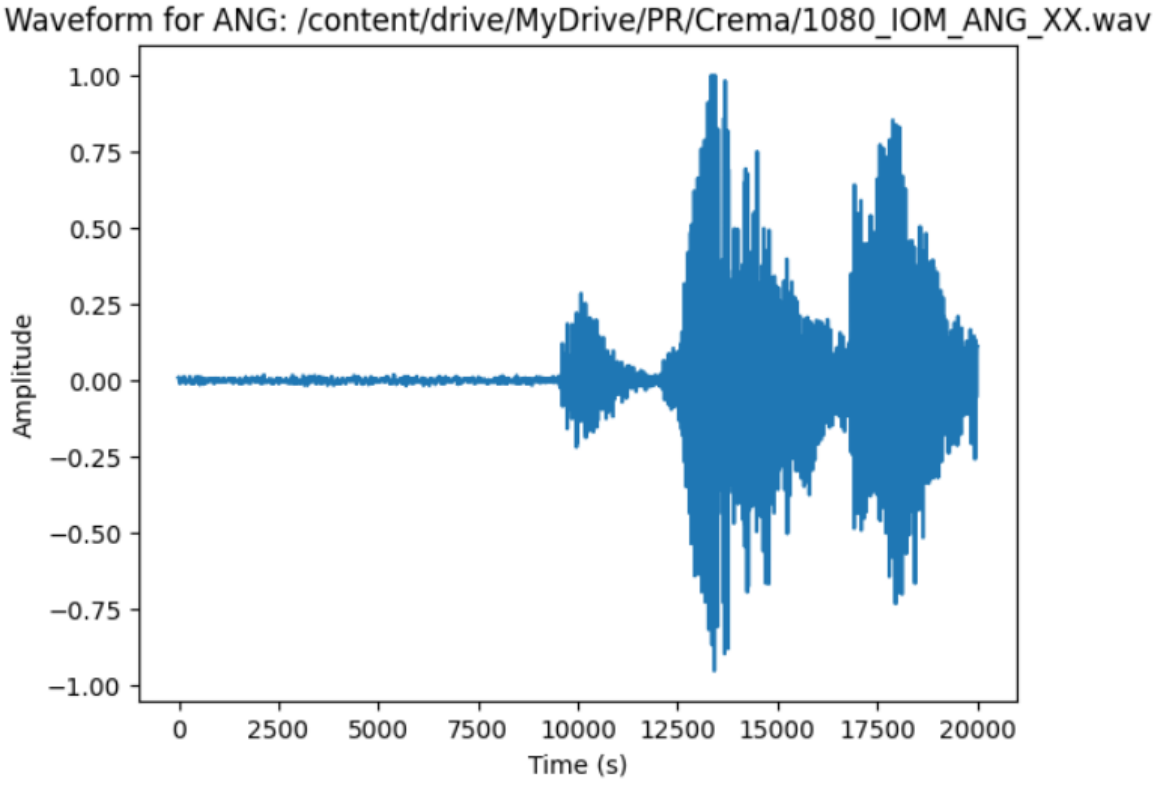

Waveform for ANG: /content/drive/MyDrive/PR/Crema/1079_TSI_ANG_XX.wav
Amplitude
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
0
2500
5000
7500
10000
12500
15000
17500
20000
Time (s)


Waveform for ANG: /content/drive/MyDrive/PR/Crema/1081_ITH_ANG_XX.wav
Amplitude
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
2500
5000
7500
10000
12500
15000
17500
20000
Time (s)


Waveform for ANG: /content/drive/MyDrive/PR/Crema/1081_TAI_ANG_XX.wav
Amplitude
Time (s)
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
0
2500
5000
7500
10000
12500
15000
17500
20000


Waveform for ANG: /content/drive/MyDrive/PR/Crema/1079_IEO_ANG_MD.wav
Amplitude
Time (s)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
2500
5000
7500
10000
12500
15000
17500
20000


Waveform for ANG: /content/drive/MyDrive/PR/Crema/1080_TIE_ANG_XX.wav
Amplitude
0.4
0.2
0.0
−0.2
−0.4
0
2500
5000
7500
10000
12500
15000
17500
20000
Time (s)


Waveform for ANG: /content/drive/MyDrive/PR/Crema/1080_IOM_ANG_XX.wav
Amplitude
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
2500
5000
7500
10000
12500
15000
17500
20000
Time (s)

```
# Retrieve the spectrogram images as resized 2D arrays
spectrogram_arrays = []
emotions = []

for index, row in data.iterrows():
    spectrogram_image = row['mel_spec_db']
    spectrogram_image_resized = spectrogram_image.resize((224, 224))  # Resize the image to a consistent shape
    spectrogram_array = np.array(spectrogram_image_resized)

    # Reshape the spectrogram array if needed
    if len(spectrogram_array.shape) > 2:
        spectrogram_array = spectrogram_array[:, :, 0]  # Take only one channel if multi-channel image

    spectrogram_arrays.append(spectrogram_array)
    emotions.append(row['emotion'])

# Convert the lists to NumPy arrays
spectrogram_arrays = np.array(spectrogram_arrays)
emotions = np.array(emotions)
```

```
from sklearn import preprocessing
from keras.utils import to_categorical
# label_encoder object knows
# how to understand word labels.
label_encoder = preprocessing.LabelEncoder()

# Encode labels in column 'species'.
emotions= label_encoder.fit_transform(emotions)
```

```
from sklearn.model_selection import train_test_split

# Splitting the data into training/validation and testing sets
X_train_val, X_test, y_train_val, y_test = train_test_split(
    [spectrogram_arrays, data['features1']], emotions, test_size=0.3, random_state=42, stratify=emotions
)

# Further split the training/validation set into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.05, random_state=42, stratify=y_train_val
)

X_train_spectrogram = X_train[0]  # Retrieve the spectrogram_arrays column from X_train
X_train_features = X_train[1]  # Retrieve the 'data['features1']' column from X_train

X_val_spectrogram = X_val[0]  # Retrieve the spectrogram_arrays column from X_val
X_val_features = X_val[1]  # Retrieve the 'data['features1']' column from X_val

X_test_spectrogram = X_test[0]  # Retrieve the spectrogram_arrays column from X_test
X_test_features = X_test[1]  # Retrieve the 'data['features1']' column from X_test
```

```
from tensorflow import keras
from tensorflow.keras import layers
#CNN for spectrograms
model = keras.Sequential([
    # Convolutional layer 1
    \
    layers.Conv2D(175, (5, 5), activation='relu', input_shape=(224, 224, 1)),

    #layers.BatchNormalization(momentum=0.9, epsilon=1e-5),
    # Max pooling layer 1
    layers.MaxPooling2D(pool_size=(5, 5)),
    # Dropout layer 1
    layers.Dropout(0.25),

    # Convolutional layer 2
    layers.Conv2D(175, (5, 5), activation='relu'),

    #layers.BatchNormalization(momentum=0.9, epsilon=1e-5),
    # Max pooling layer 2
    layers.MaxPooling2D(pool_size=(2, 2)),


    # Dropout layer 2
    layers.Dropout(0.25),

    # Flatten layer
    layers.Flatten(),

    # Dense layer 1
    layers.Dense(45, activation='relu'),
    # Dropout layer 3
    layers.Dropout(0.25),
```

```
    # Dense layer 2
    layers.Dense(6, activation='softmax')
])

model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 220, 220, 175)     4550

 max_pooling2d (MaxPooling2D  (None, 44, 44, 175)      0
 )

 dropout_4 (Dropout)         (None, 44, 44, 175)       0

 conv2d_1 (Conv2D)           (None, 40, 40, 175)       765800

 max_pooling2d_1 (MaxPooling  (None, 20, 20, 175)      0
 2D)

 dropout_5 (Dropout)         (None, 20, 20, 175)       0

 flatten (Flatten)           (None, 70000)             0

 dense_3 (Dense)             (None, 45)                3150045

 dropout_6 (Dropout)         (None, 45)                0

 dense_4 (Dense)             (None, 6)                 276

=================================================================
Total params: 3,920,671
Trainable params: 3,920,671
Non-trainable params: 0
```

```
[ ] checkpoint_callback = ModelCheckpoint(
        filepath='best_model.h5',
        monitor='val_acc',  # Metric to monitor (e.g., validation accuracy)
        save_best_only=True,  # Save only the best model
        mode='max',  # Mode of the monitored metric (e.g., 'max' for accuracy)
        verbose=1  # Optional: Print messages when saving the best model
    )
```

```
# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(X_train_spectrogram, y_train, epochs=10, batch_size=32, validation_data=(X_val_spectrogram,y_val),callbacks=[checkpoint_callback])
```

```
Epoch 1/10
21/21 [==============================] - ETA: 0s - loss: 17.4864 - accuracy: 0.2617WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 9s 265ms/step - loss: 17.4864 - accuracy: 0.2617 - val_loss: 1.6046 - val_accuracy: 0.3429
Epoch 2/10
21/21 [==============================] - ETA: 0s - loss: 1.6208 - accuracy: 0.3278WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 3s 146ms/step - loss: 1.6208 - accuracy: 0.3278 - val_loss: 1.6294 - val_accuracy: 0.4286
Epoch 3/10
21/21 [==============================] - ETA: 0s - loss: 1.5666 - accuracy: 0.3624WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 3s 142ms/step - loss: 1.5666 - accuracy: 0.3624 - val_loss: 1.5591 - val_accuracy: 0.4000
Epoch 4/10
21/21 [==============================] - ETA: 0s - loss: 1.5445 - accuracy: 0.4015WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 3s 145ms/step - loss: 1.5445 - accuracy: 0.4015 - val_loss: 1.5946 - val_accuracy: 0.3143
Epoch 5/10
21/21 [==============================] - ETA: 0s - loss: 1.5033 - accuracy: 0.4195WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 3s 144ms/step - loss: 1.5033 - accuracy: 0.4195 - val_loss: 1.5041 - val_accuracy: 0.4571
Epoch 6/10
21/21 [==============================] - ETA: 0s - loss: 1.4424 - accuracy: 0.4602WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 3s 142ms/step - loss: 1.4424 - accuracy: 0.4602 - val_loss: 1.5177 - val_accuracy: 0.4286
Epoch 7/10
21/21 [==============================] - ETA: 0s - loss: 1.4142 - accuracy: 0.4827WARNING:tensorflow:Can save best model only with val_acc available, skipping.
```

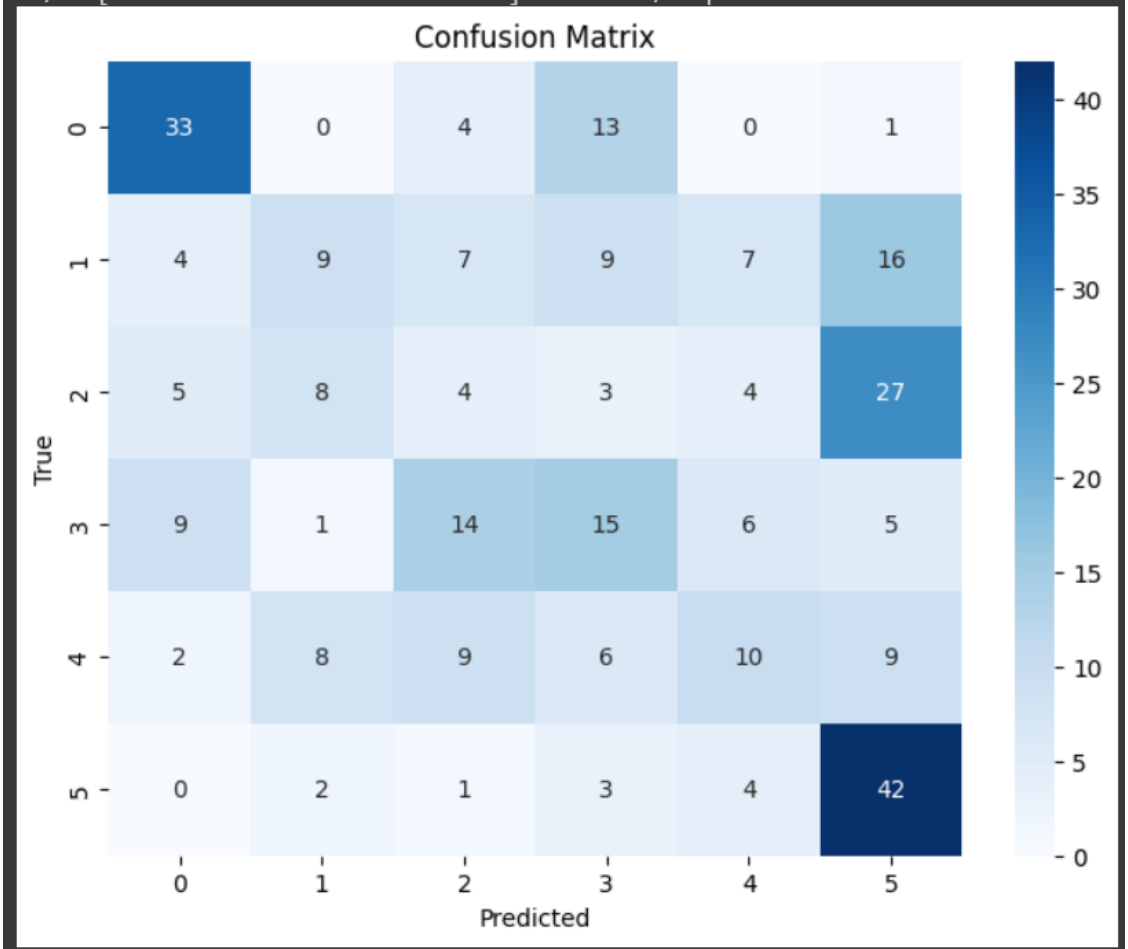
```
Epoch 7/10
21/21 [==============================] - ETA: 0s - loss: 1.4142 - accuracy: 0.4827WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 3s 141ms/step - loss: 1.4142 - accuracy: 0.4827 - val_loss: 1.4482 - val_accuracy: 0.4571
Epoch 8/10
21/21 [==============================] - ETA: 0s - loss: 1.3899 - accuracy: 0.4602WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 3s 143ms/step - loss: 1.3899 - accuracy: 0.4602 - val_loss: 1.5644 - val_accuracy: 0.4286
Epoch 9/10
21/21 [==============================] - ETA: 0s - loss: 1.3542 - accuracy: 0.4977WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 3s 143ms/step - loss: 1.3542 - accuracy: 0.4977 - val_loss: 1.5721 - val_accuracy: 0.3714
Epoch 10/10
21/21 [==============================] - ETA: 0s - loss: 1.2921 - accuracy: 0.5398WARNING:tensorflow:Can save best model only with val_acc available, skipping.
21/21 [==============================] - 3s 145ms/step - loss: 1.2921 - accuracy: 0.5398 - val_loss: 1.5711 - val_accuracy: 0.2857
```

```
from sklearn.metrics import f1_score, precision_score, recall_score, confusion_matrix
y_pred1 = model.predict(X_test_spectrogram)
y_pred = np.argmax(y_pred1, axis=1)

# Print f1, precision, and recall scores
print(precision_score(y_test, y_pred , average="macro"))
print(recall_score(y_test, y_pred , average="macro"))
print(f1_score(y_test, y_pred , average="macro"))
```

```
10/10 [==============================] - 1s 62ms/step
0.3492228795945864
0.37225535902006496
0.3451388036914353
```

```python
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# Make predictions using the trained CNN model
y_pred = model.predict(X_test_spectrogram)
y_pred_classes = np.argmax(y_pred, axis=1)

# Compute the confusion matrix
cm = confusion_matrix(y_test, y_pred_classes)

# Plot the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
emotions = ['ANG', 'DIS', 'FEA', 'HAP', 'NEU', 'SAD']
# Identify the most confusing classes based on the confusion matrix
np.fill_diagonal(cm, 0)  # Remove correct predictions from the confusion matrix
most_confusing_classes = np.unravel_index(np.argmax(cm), cm.shape)
class1 = emotions[most_confusing_classes[0]]
class2 = emotions[most_confusing_classes[1]]
print(f'The most confusing classes are: {emotions[most_confusing_classes[0]]} and {emotions[most_confusing_classes[1]]}.')
```

```
10/10 [==============================] - 0s 28ms/step
```




```
The most confusing classes are: FEA and SAD.
```

```
from sklearn.model_selection import train_test_split
np.fill_diagonal(cm, 0)  # Remove correct predictions from the confusion matrix
most_confusing_classes = np.unravel_index(np.argmax(cm), cm.shape)
class1 = emotions[most_confusing_classes[0]]
class2 = emotions[most_confusing_classes[1]]

print(f': {class1} and {class2}  are the most confusing classes.')
```

```
: 0 and 0  are the most confusing classes.
```

```
# Split the data into training/validation and testing sets
X_train_val, X_test, y_train_val, y_test = train_test_split(data['features1'], data['emotion'], test_size=0.3, random_state=42, stratify=data['emotion'])

# print(type(np.array(X_train_val.iloc[3])[0]))

# Further split the training/validation set into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val, test_size=0.05, random_state=42, stratify=y_train_val)
```

```
#best 23
from keras.layers import Input
from keras.layers import Conv1D, MaxPool1D, Dropout, GlobalMaxPooling1D, Dense
from keras import activations
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras import losses

from tensorflow.keras import optimizers

nclass = 6
inp = Input(shape=(41920, 1))
img_1 = Conv1D(16, kernel_size=9, activation=activations.relu, padding="valid")(inp)
img_1 = Conv1D(16, kernel_size=9, activation=activations.relu, padding="valid")(img_1)
img_1 = MaxPool1D(pool_size=16)(img_1)
img_1 = Dropout(rate=0.1)(img_1)
img_1 = Conv1D(32, kernel_size=3, activation=activations.relu, padding="valid")(img_1)
img_1 = Conv1D(32, kernel_size=3, activation=activations.relu, padding="valid")(img_1)
img_1 = MaxPool1D(pool_size=4)(img_1)
img_1 = Dropout(rate=0.1)(img_1)
img_1 = Conv1D(32, kernel_size=3, activation=activations.relu, padding="valid")(img_1)
img_1 = Conv1D(32, kernel_size=3, activation=activations.relu, padding="valid")(img_1)
img_1 = MaxPool1D(pool_size=4)(img_1)
img_1 = Dropout(rate=0.1)(img_1)
img_1 = Conv1D(256, kernel_size=3, activation=activations.relu, padding="valid")(img_1)
img_1 = Conv1D(256, kernel_size=3, activation=activations.relu, padding="valid")(img_1)
img_1 = GlobalMaxPooling1D()(img_1)
img_1 = Dropout(rate=0.2)(img_1)

dense_1 = Dense(64, activation=activations.relu)(img_1)
dense_1 = Dense(1028, activation=activations.relu)(dense_1)
dense_1 = Dense(nclass, activation=activations.softmax)(dense_1)
```

```
model = models.Model(inputs=inp, outputs=dense_1)
opt = optimizers.Adam(0.0001)

model.compile(optimizer=opt, loss=losses.sparse_categorical_crossentropy, metrics=['acc'])
model.summary()
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, 41920, 1)]        0

 conv1d (Conv1D)             (None, 41912, 16)         160

 conv1d_1 (Conv1D)           (None, 41904, 16)         2320

 max_pooling1d (MaxPooling1D  (None, 2619, 16)         0
 )

 dropout (Dropout)           (None, 2619, 16)          0

 conv1d_2 (Conv1D)           (None, 2617, 32)          1568

 conv1d_3 (Conv1D)           (None, 2615, 32)          3104

 max_pooling1d_1 (MaxPooling  (None, 653, 32)          0
 1D)

 dropout_1 (Dropout)         (None, 653, 32)           0

 conv1d_4 (Conv1D)           (None, 651, 32)           3104

 conv1d_5 (Conv1D)           (None, 649, 32)           3104

 max_pooling1d_2 (MaxPooling  (None, 162, 32)          0
 1D)
```

```
 dropout_2 (Dropout)         (None, 162, 32)           0

 conv1d_6 (Conv1D)           (None, 160, 256)          24832

 conv1d_7 (Conv1D)           (None, 158, 256)          196864

 global_max_pooling1d (Globa  (None, 256)              0
 lMaxPooling1D)

 dropout_3 (Dropout)         (None, 256)               0

 dense (Dense)               (None, 64)                16448

 dense_1 (Dense)             (None, 1028)              66820

 dense_2 (Dense)             (None, 6)                 6174

=================================================================
Total params: 324,498
Trainable params: 324,498
Non-trainable params: 0

_________________________________________________________________
```

```
X_train = np.array(X_train)
y_train = np.array(y_train)
X_val = np.array(X_val)
y_val = np.array(y_val)
new_val=np.zeros((X_val.shape[0],41920))
i=0
for tupl in X_val:
  tupl=tupl.replace('\n', '')
  tupl=tupl.replace(' ', '')
  tup= tupl.strip('[]').split(',')
  new_val[i]=np.array(tup)
  i=i+1
```

```
new_val = np.array([[float(element) for element in row] for row in new_val])
new_y_val= np.array(y_val).reshape(-1, 1)
new_y_val = np.array([[float(element) for element in row] for row in new_y_val])


new_train=np.zeros((X_train.shape[0],41920))
i=0
for tupl in X_train:
  tupl=tupl.replace('\n', '')
  tupl=tupl.replace(' ', '')
  tup= tupl.strip('[]').split(',')
  new_train[i]=np.array(tup)
  i=i+1
new_train = np.array([[float(element) for element in row] for row in new_train])
new_y_train= np.array(y_train).reshape(-1, 1)
new_y_train = np.array([[float(element) for element in row] for row in new_y_train])
```

```
from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint_callback = ModelCheckpoint(
    filepath='best_model.h5',
    monitor='val_acc',  # Metric to monitor (e.g., validation accuracy)
    save_best_only=True,  # Save only the best model
    mode='max',  # Mode of the monitored metric (e.g., 'max' for accuracy)
    verbose=1  # Optional: Print messages when saving the best model
)
model.fit(new_train, new_y_train, epochs=1000, validation_data=(new_val, new_y_val),callbacks=[checkpoint_callback])
```

```
X_test = np.array(X_test)
y_test = np.array(y_test)
new_test=np.zeros((X_test.shape[0],41920))
i=0
for tupl in X_test:
  tupl=tupl.replace('\n', '')
  tupl=tupl.replace(' ', '')
  tup= tupl.strip('[]').split(',')
  new_test[i]=np.array(tup)
  i=i+1
new_test = np.array([[float(element) for element in row] for row in new_test])
new_y_test= np.array(y_test).reshape(-1, 1)
new_y_test = np.array([[float(element) for element in row] for row in new_y_test])
```

```
y_pred = model.predict(new_test)
```

```
10/10 [==============================] - 1s 46ms/step
```

```
y_pred_labels = np.argmax(y_pred, axis=1)
y_test_labels = new_y_test.T.astype(int).reshape(-1)
print(y_pred_labels)
print(y_test_labels)
```

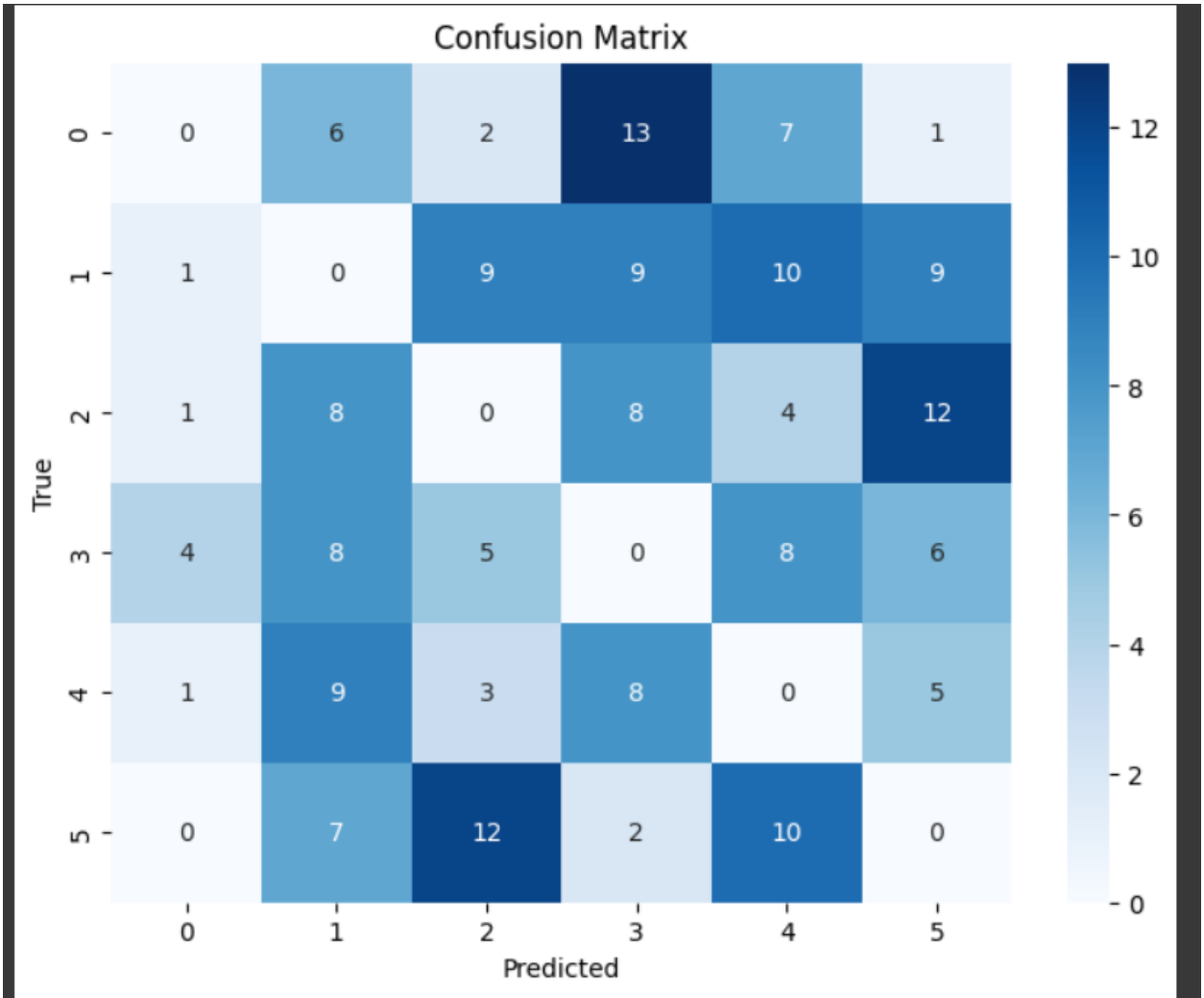
```
y_pred_labels = np.argmax(y_pred, axis=1)
y_test_labels = new_y_test.T.astype(int).reshape(-1)
print(y_pred_labels)
print(y_test_labels)
```

```
[3 4 0 2 5 2 5 1 1 0 5 4 0 1 5 3 3 4 0 1 5 5 4 5 4 3 1 1 3 2 3 3 5 2 3 4 4
 0 5 1 1 5 5 5 1 3 4 2 5 4 2 4 4 1 5 4 1 5 3 5 4 2 1 5 1 4 3 2 3 0 4 2 3 0
 4 4 3 0 4 4 1 1 0 2 3 4 4 4 1 2 1 3 3 5 1 4 3 0 1 2 2 1 4 3 5 4 3 3 4 3 4
 3 1 3 1 0 2 1 3 2 1 1 1 0 0 3 3 4 1 4 3 3 3 4 2 0 5 2 3 2 4 5 2 3 2 5 5 4
 2 0 2 5 4 5 4 4 4 3 3 5 1 1 5 5 2 3 2 3 1 1 0 5 2 0 2 2 5 4 4 2 0 2 4 2 4
 0 1 0 2 2 4 4 3 2 5 2 3 4 2 1 0 5 3 5 0 1 2 5 2 1 5 2 3 5 5 3 5 5 4 1 3 2
 2 1 4 3 5 1 3 3 5 3 0 3 3 5 1 1 3 2 1 3 1 3 4 4 1 3 3 1 5 1 5 4 5 3 1 0 0
 4 2 5 4 4 3 4 2 1 5 0 3 4 5 1 5 1 5 1 0 5 5 0 1 4 2 4 4 2 5 5 2 1 2 3 0 2
 2 3 4 3]
[3 2 0 2 2 2 2 3 0 4 5 3 0 5 2 4 2 5 0 3 5 3 1 3 4 3 1 1 3 4 0 4 5 3 1 4 3
 0 2 2 1 4 1 4 3 3 2 2 5 5 5 4 4 1 5 4 4 2 3 5 4 1 4 1 4 0 3 5 1 0 1 2 1 0
 5 0 0 0 4 3 2 2 3 1 0 5 2 0 0 1 1 3 1 5 5 3 3 0 1 2 4 1 4 4 2 4 2 3 4 0 1
 3 5 0 0 0 2 3 3 2 3 3 2 0 0 3 0 0 5 1 1 5 2 5 5 0 2 2 0 3 5 1 5 4 2 4 5 0
 2 1 2 1 1 0 4 2 1 0 0 5 1 4 5 1 1 2 5 0 0 2 3 1 5 0 4 3 5 0 3 5 0 1 4 2 3
 0 2 2 3 1 1 4 3 5 5 2 4 4 5 1 0 5 2 2 3 1 3 2 0 3 2 2 2 2 5 1 4 5 3 1 2 2
 2 1 4 3 5 5 4 0 1 4 3 3 3 2 1 1 1 1 4 3 5 2 1 5 0 1 3 5 5 2 3 1 5 0 4 0 0
 5 1 5 4 4 1 5 2 4 3 0 4 4 5 4 4 2 5 0 0 3 3 0 3 5 1 0 1 2 1 1 0 4 5 5 0 5
 5 3 3 0]
```

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test_labels, y_pred_labels)

# Calculate F1 score
f1 = f1_score(y_test_labels, y_pred_labels, average='weighted')

# Calculate confusion matrix
confusion_mat = confusion_matrix(y_test_labels, y_pred_labels)

print("Accuracy:", accuracy)
print("F1 Score:", f1)
print("Confusion Matrix:")
print(confusion_mat)
```

```
Accuracy: 0.37333333333333335
F1 Score: 0.38042711520762607
Confusion Matrix:
[[22  6  2 13  7  1]
 [ 1 14  9  9 10  9]
 [ 1  8 18  8  4 12]
 [ 4  8  5 19  8  6]
 [ 1  9  3  8 18  5]
 [ 0  7 12  2 10 21]]
```

```python
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# Make predictions using the trained CNN model
#y_pred = model.predict(X_test)
#y_pred_classes = np.argmax(y_pred, axis=1)

# Compute the confusion matrix
#cm = confusion_matrix(y_test, y_pred_classes)

# Plot the confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_mat, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()

# Identify the most confusing classes based on the confusion matrix
np.fill_diagonal(confusion_mat, 0)  # Remove correct predictions from the confusion matrix
most_confusing_classes = np.unravel_index(np.argmax(confusion_mat), confusion_mat.shape)
class1 = emotions[most_confusing_classes[0]]
class2 = emotions[most_confusing_classes[1]]
print(f'The most confusing classes are: {class1} and {class2}.')
```


Confusion Matrix
True
Predicted
0 6 2 13 7 1
1 0 9 9 10 9
1 8 0 8 4 12
4 8 5 0 8 6
1 9 3 8 0 5
0 7 12 2 10 0
0 1 2 3 4 5
12 10 8 6 4 2 0
The most confusing classes are: ANG and HAP.

- **Reference Paper:**

We used an article as our reference for building the architecture of the model which is called “Recognition of Emotions in Speech Using Convolutional Neural Networks on Different Datasets”

by Marta Zielonka, Artur Piastowski, Andrzej Czyzewski, Paweł Nadachowski, Maksymilian Operlejn and Kamil Kaczor

The study focuses on applying Convolutional Neural Networks (CNN) to extract emotions from spectrograms and mel-spectrograms. The research compares the efficiency of these two feature extraction methods in representing emotions. The study uses five popular datasets: CREMA-D, RAVDESS, SAVEE, TESS, and IEMOCAP, and considers six emotion classes: happiness, anger, sadness, fear, disgust, and neutral.

The results show that mel-spectrograms are better suited for training CNN-based Speech Emotion Recognition (SER) models. An accuracy of 55.89% is achieved when recognizing four emotions, and the most accurate model for six emotion recognition achieves 57.42% accuracy using a combination of four datasets.

The article also addresses the issue of inappropriate data division between training and test sets, which can significantly impact the test accuracy of CNNs. The popular ResNet18 architecture is employed to validate the research results and demonstrate that the data division problem is not specific to custom CNN architectures.
Additionally, the article includes a study on the label correctness of the CREMA-D dataset, which involves the use of a questionnaire.